```
import numpy as np

class GridWorldMDP:
    def __init__(self, size, goal, trap):
        self.size = size
        self.goal = goal
        self.trap = trap
        self.state_space = [(i, j) for i in range(size) for j in range(size)]
        self.action_space = ['UP', 'DOWN', 'LEFT', 'RIGHT']
        self.transitions = self.build_transitions()
        self.rewards = self.build_rewards()

    def build_transitions(self):
        transitions = {}
        for state in self.state_space:
            transitions[state] = {}
            for action in self.action_space:
                transitions[state][action] = self.calculate_transitions(state, action)
        return transitions

    def calculate_transitions(self, state, action):
        i, j = state
        if action == 'UP':
            return self.validate_state(i - 1, j)
        elif action == 'DOWN':
            return self.validate_state(i + 1, j)
        elif action == 'LEFT':
            return self.validate_state(i, j - 1)
        elif action == 'RIGHT':
            return self.validate_state(i, j + 1)

    def validate_state(self, i, j):
        i = max(0, min(i, self.size - 1))
        j = max(0, min(j, self.size - 1))

        # Terminal states stay in same state
        if (i, j) == self.goal or (i, j) == self.trap:
            return [(1.0, (i, j))]
        else:
            return [(1.0, (i, j))]

    def build_rewards(self):
        rewards = {}
        for state in self.state_space:
            rewards[state] = -1.0
        rewards[self.goal] = 0.0
        rewards[self.trap] = -10.0
        return rewards


# ---------------- VALUE ITERATION ----------------
def value_iteration(mdp, gamma=0.9, epsilon=0.01):
    state_values = {state: 0.0 for state in mdp.state_space}

    while True:
        delta = 0
        for state in mdp.state_space:
            if state == mdp.goal or state == mdp.trap:
                continue

            v = state_values[state]
```

```
            state_values[state] = max(
                sum(p * (mdp.rewards[next_state] + gamma * state_values[next_state])
                    for p, next_state in mdp.transitions[state][action])
                for action in mdp.action_space
            )

            delta = max(delta, abs(v - state_values[state]))

        if delta < epsilon:
            break

    return state_values


# ---------------- POLICY ITERATION ----------------
def policy_iteration(mdp, gamma=0.9):
    # Initialize random policy
    policy = {
        state: np.random.choice(mdp.action_space)
        for state in mdp.state_space
        if state != mdp.goal and state != mdp.trap
    }

    state_values = {state: 0.0 for state in mdp.state_space}

    while True:

        # -------- Policy Evaluation --------
        while True:
            delta = 0
            for state in mdp.state_space:
                if state == mdp.goal or state == mdp.trap:
                    continue

                v = state_values[state]
                action = policy[state]

                state_values[state] = sum(
                    p * (mdp.rewards[next_state] + gamma * state_values[next_state])
                    for p, next_state in mdp.transitions[state][action]
                )

                delta = max(delta, abs(v - state_values[state]))

            if delta < 0.01:
                break

        # -------- Policy Improvement --------
        policy_stable = True

        for state in mdp.state_space:
            if state == mdp.goal or state == mdp.trap:
                continue

            old_action = policy[state]

            best_action = max(
                mdp.action_space,
                key=lambda a: sum(
                    p * (mdp.rewards[next_state] + gamma * state_values[next_state])
                    for p, next_state in mdp.transitions[state][a]
                )
            )
```

```
                policy[state] = best_action

                if old_action != best_action:
                    policy_stable = False

            if policy_stable:
                break

        return policy, state_values


    # ---------------- MAIN ----------------
    if __name__ == "__main__":
        size = 3
        goal = (2, 2)
        trap = (1, 1)

        mdp = GridWorldMDP(size, goal, trap)

        # Value Iteration
        vi_values = value_iteration(mdp)
        print("Value Iteration Results:")
        for state, value in vi_values.items():
            print(f"State: {state}, Value: {round(value, 2)}")

        # Policy Iteration
        policy, pi_values = policy_iteration(mdp)
        print("\nPolicy Iteration Results:")
        for state in policy:
            print(f"State: {state}, Best Action: {policy[state]}")
```

```
Value Iteration Results:
State: (0, 0), Value: -2.71
State: (0, 1), Value: -1.9
State: (0, 2), Value: -1.0
State: (1, 0), Value: -1.9
State: (1, 1), Value: 0.0
State: (1, 2), Value: 0.0
State: (2, 0), Value: -1.0
State: (2, 1), Value: 0.0
State: (2, 2), Value: 0.0

Policy Iteration Results:
State: (0, 0), Best Action: DOWN
State: (0, 1), Best Action: RIGHT
State: (0, 2), Best Action: DOWN
State: (1, 0), Best Action: DOWN
State: (1, 2), Best Action: DOWN
State: (2, 0), Best Action: RIGHT
State: (2, 1), Best Action: RIGHT
```